// Day 6 of 10

Day 6

10 Clean Code Rules

Developers make these every day.
Don't be one of them.

→ Swipe through all 10

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #01

Meaningful Names

Code is read more than written

```
// Bad
int d;
String n;
void proc(int x){ ... }

// Good
int daysSinceCreation;
String customerName;
void calculateOrderTotal(int qty){ }
```

💡 If you need a comment, rename it

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #02

Functions Do ONE Thing

Single responsibility per function

```
// Bad
void processUser(User u){
  validateUser(u);
  saveToDatabase(u);
  sendWelcomeEmail(u);
  logActivity(u);
}

// Good
void onboardUser(User u){
  validate(u);
  save(u);
```

💡 If 'and' fits in description, split it

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #03

Max 3 Function Params

Too many params = bad design

```
// Bad
void createOrder(String name,
  String addr, int qty, double price,
  String status, Long userId){ ... }

// Good
void createOrder(OrderRequest req){
  // All data in one clean object
}
```

💡 Group params into a request object

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #04

No Magic Numbers

Numbers without context = confusing

```
// Bad
if(status == 3){ ... }
if(age > 18){ ... }
Thread.sleep(86400000);

// Good
if(status == ORDER_CONFIRMED){ ... }
if(age > LEGAL_AGE){ ... }
Thread.sleep(ONE_DAY_IN_MS);
```

💡 Extract constants with clear names

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #05

DRY Principle

Duplication = multiple places to fix bugs

```
// Bad
// Same validation in 3 controllers
if(email == null || email.isEmpty()){
  throw new ValidationException( ... );
}

// Good
private void validateEmail(String email){
  if(isBlank(email)) throw new ... ;
}
// Called from everywhere
```

💡 If you copy-paste, extract a method

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #06

Short Functions

If you scroll, it's too long

```
// Bad
public void doEverything(){
  // 200 lines of code
  // multiple responsibilities
}

// Good
public void process(){
  validate();
  transform();
  save();
  notify();
```

💡 Aim for functions under 20 lines

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #07

Comments Explain WHY

Code shows what, comments show why

```
// Bad
// Loop through users
for(User u : users){
  // check if active
  if(u.isActive()){ ... }
}

// Good
// Skip inactive users to avoid
// emailing cancelled accounts
for(User u : users){
  if(u.isActive()){ ... }
```

💡 Good code needs very few comments

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #08

Guard Clauses Early

Reduce nesting with early returns

```
// Bad
void process(User u){
  if(u ≠ null){
    if(u.isActive()){
      if(u.hasPermission()){
        // actual logic
      }
    }
  }
}

// Good
```

💡 Fail fast = flatter code

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #09

Consistent Formatting

Inconsistency makes code unreadable

```
// Bad
public void save(User u){if(u≠null){
repo.save(u);}
}

// Good
public void save(User u) {
  if (u == null) return;
  repo.save(u);
}
```

💡 Use IntelliJ auto-format: Ctrl+Alt+L

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #10

Tests Are Documentation

Good tests describe expected behavior

```
// Bad
@Test void test1(){ ... }
@Test void testSomething(){ ... }

// Good
@Test
void shouldReturn404WhenUserNotFound(){ }
@Test
void shouldCreateOrderForActiveUser(){ }
```

💡 Test name = specification of behavior

Shubham Bhati
linkedin.com/in/bhatishubham

```
done = true
```

Follow for more tips!

Like · Comment · Repost

Shubham Bhati
linkedin.com/in/bhatishubham